

String Based Problem statements-2

Problem 1. Recover the Missing Character

A detective has an incomplete word containing exactly one underscore (_) and a colon-separated dictionary. Find all dictionary words that match the incomplete word. The underscore can represent any single character. Ignore letter case, preserve dictionary order, and return matching words in uppercase.

Input format

- Line 1: Incomplete word.
- Line 2: Colon-separated dictionary words.

Output format

Matching words separated by colons in uppercase, or ERROR-009.

Constraints

- 1 <= N <= 1000
- Word length: 1–100.
- Exactly one underscore appears in the pattern.
- Dictionary words contain English letters only.

Example

Input

Fi_er
Fever:filer:Filter:Fixer:fiber:fibre:tailor:offer

Output

FILER:FIXER:FIBER

Explanation: The candidate must have five characters, with F, I, and E in their specified positions. filer, fixer, and fiber satisfy the pattern.

Problem 2. Replace Every Missing Character

A document-recovery system receives a word pattern containing multiple underscores. Each underscore represents exactly one unknown character. Find all dictionary words that match every fixed character at the correct position, ignoring case.

Unlike Problem 1, the pattern can contain any number of underscores. Preserve the dictionary order and return uppercase results.

Input format

- Line 1: Pattern containing one or more underscores.
- Line 2: Colon-separated dictionary words.

Output format

Matching words separated by colons, or NO-MATCH.



Constraints

- 1 <= N <= 2000
- Pattern length: 1–100.
- At least one underscore appears in the pattern.
- Words contain English letters only.

Example

Input
c__e
care:code:cake:cope:case:cheese
Output
CARE:CODE:CAKE:COPE:CASE

Explanation: Every candidate must have four characters, start with c, and end with e. The two underscores can represent any two characters.

Problem 3. Find Words with the Highest Character Frequency

A text-analysis application receives a list of words and a target character. Identify all words containing the target character the maximum number of times among the supplied words. Ignore case while counting. Return all tied words in their original order, converted to uppercase.
Words containing zero occurrences are eligible only if the maximum frequency is zero.

Input format

- Line 1: A single English letter.
- Line 2: Colon-separated words.

Output format

Words with the highest target-character frequency, separated by colons, or INVALID-INPUT if the word list is empty.

Constraints

- 1 <= N <= 1000
- Word length: 1–100.
- Target contains exactly one English letter.
- Matching is case-insensitive.

Example

Input
a

banana:apple:grape:pear:avocado

Output
BANANA:AVOCADO

Explanation: banana contains three occurrences of a, and avocado also contains three. The other words contain fewer occurrences.

Problem 4. Filter Products by Multiple Conditions

An online store maintains product names in a colon-separated list. The user specifies a required prefix, a required suffix, and a minimum length. Return products that satisfy all three conditions. Comparisons must ignore case, and output must be uppercase in the original order.

Input format

- Line 1: Required prefix.
- Line 2: Required suffix.
- Line 3: Minimum length L.
- Line 4: Colon-separated product names.

Output format
Matching product names separated by colons, or NO-PRODUCT.

Constraints

- 1 <= N <= 2000
- Product-name length: 1–100.
- 1 <= L <= 100
- Prefix and suffix contain letters only.

Example
Input
pro
x
5
ProductX:ProMax:ProBox:ProjectX:ProX

Output
PRODUCTX:PROJECTX

Explanation: The product must begin with pro, end with x, and contain at least five characters. ProMax and ProBox fail the suffix condition.



Problem 5. Normalize and Compare Sentences

A plagiarism-checking application receives two sentences. It must determine whether they contain the same sequence of words after normalization. Ignore letter case and repeated spaces. Punctuation characters are not present in the input. If the normalized sentences are identical, print MATCH; otherwise, print NOT-MATCH.

Input format

- Line 1: First sentence.
- Line 2: Second sentence.

Output format

MATCH or NOT-MATCH.

Constraints

- Each sentence contains at most 1000 characters.
- Each sentence contains at least one word.
- Words contain English letters only.
- Words must remain in the same order.

Example

Input

Java is Easy
java is easy

Output

MATCH

Explanation: After ignoring case and collapsing repeated spaces, both sentences become java is easy.

Problem 6. Find the First Non-Repeating Character

A logging system processes an event identifier and needs to identify its first non-repeating character. Count character occurrences without considering case, but return the character in its original case. If every character repeats, return NO-UNIQUE-CHARACTER.

Input format

Line 1: An alphanumeric string.

Output format

Print the first non-repeating character, or NO-UNIQUE-CHARACTER.

Constraints

- String length: 1–100,000.
- The string contains English letters and digits.

- Letter case is ignored when counting.

Example

Input
aAbBCdD

Output
C

Explanation: a, b, and d each occur twice when case is ignored. C is the first character in the original string whose case-insensitive frequency is one.

Problem 7. Sort Words by Length and Alphabetical Order

A dictionary application receives a list of words. Sort them by ascending length. If two words have equal lengths, sort them alphabetically without considering case. When words are identical after case normalization, preserve their original order. Output every word in uppercase.

Input format

- Line 1: Integer N.
- Line 2: N space-separated words.

Output format

Sorted words separated by spaces.

Constraints

- 1 <= N <= 10000
- Word length: 1–100.
- Words contain English letters only.

Example

Input
6
pear fig Apple bat kiwi ant

Output
ANT BAT FIG PEAR KIWI APPLE

Explanation: ant, bat, and fig have three characters and are sorted alphabetically. pear and kiwi have four characters and are sorted alphabetically, followed by apple with five.



Problem 8. Search Words Using Multiple Queries

A dictionary service receives multiple search patterns and one shared dictionary. Each pattern contains exactly one underscore. For each query, find all matching words, ignoring case. Preserve dictionary order and print each query's result independently.

Input format

- Line 1: Number of queries Q.
- Next Q lines: Search patterns.
- Final line: Colon-separated dictionary.

Output format

One line per query, containing matching words separated by colons, or ERROR-009.

Constraints

- 1 <= Q <= 100
- Dictionary size: 1–2000 words.
- Word length: 1–100.
- Each pattern contains exactly one underscore.

Example

Input

3
c_t
b_g
d_g
cat:cut:cot:bag:beg:dog:dig

Output

CAT:CUT:COT
BAG:BEG
DOG:DIG

Explanation: Each query is evaluated independently against the same dictionary.

Problem 9. Find Anagrams of a Target Word

A word-game application receives a target word and a list of candidates. A candidate is an anagram if it contains exactly the same characters with the same frequencies, regardless of order or letter case. Return all matching candidates in uppercase, preserving their original order. Duplicate candidates must remain in the output.

Input format

- Line 1: Target word.

- Line 2: Colon-separated candidates.

Output format
Matching anagrams separated by colons, or NO-ANAGRAM.

Constraints
- Target length: 1–1000.
- Number of candidates: 1–5000.
- Candidate length: 1–1000.
- Words contain English letters only.

Example

Input
listen
Listen:silent:enlist:tinsel:google:inlets

Output
LISTEN:SILENT:ENLIST:TINSEL:INLETS

Explanation: Each returned word contains exactly the same character frequencies as listen.

Problem 10. Remove Duplicate Words While Preserving Order

A text-processing application receives a colon-separated sequence of words. Remove duplicate words without considering case. Keep the first occurrence of each distinct word and preserve the original order. Convert all retained words to uppercase.

Input format
Line 1: Colon-separated words.

Output format
Unique words separated by colons.
Constraints
- Number of words: 1–100,000.
- Word length: 1–100.
- Words contain English letters only.

Example

Input
Java:Python:JAVA:C:python:JavaScript:C

Output

JAVA:PYTHON:C:JAVASCRIPT

Explanation: The later occurrences of Java, Python, and C are removed because each word has already appeared.

Problem 11. Find the Minimum Edits to Match a Dictionary Word

A spell-checking system receives a misspelled word and a dictionary. For each dictionary word, calculate the minimum number of single-character edits required to transform the input word into that dictionary word. An edit can be an insertion, deletion, or substitution.
Return all dictionary words with the smallest edit distance, in their original order and uppercase. Matching is case-insensitive. If multiple dictionary words have the same minimum distance, return all of them.

Input format

- Line 1: Input word.
- Line 2: Colon-separated dictionary words.

Output format

Closest words in uppercase, separated by colons, or NO-WORD.

Constraints

- Input and candidate word lengths: 1–500.
- Dictionary size: 1–1000.
- Words contain English letters only.
- Ignore case when calculating edit distance.

Example

Input

cat
bat:cut:cart:dog

Output

BAT:CUT
Explanation: Replacing c with b transforms cat into bat in one edit. Replacing a with u transforms cat into cut in one edit. cart requires an insertion, while dog requires three substitutions.

Problem 12. Rank Words by Frequency and First Appearance

A search analytics application receives a sequence of words and must rank the distinct words by their occurrence frequency. Case differences are ignored. Words with higher frequencies appear first. If two words have the same frequency, the word that first appeared earlier in the input comes first.

Input format

- Line 1: Integer N.
- Line 2: N space-separated words.

Output format
Print the distinct words in ranked order, converted to uppercase and separated by spaces.

Constraints
- 1 <= N <= 100,000
- Word length: 1–100.
- Words contain English letters only.

Example

Input
8
java python JAVA c python java c sql

Output
JAVA PYTHON C SQL

Explanation: java appears three times, python twice, c twice, and sql once. PYTHON appears before C because it first appeared earlier in the input.

Problem 13. Find the Longest Dictionary Word That Matches a Subsequence

A text editor suggests words that can be formed by deleting zero or more characters from a given input string without changing the order of the remaining characters. Given an input string and a dictionary, identify the longest dictionary word that is a subsequence of the input.
If multiple words have the same maximum length, return the word that appears first in the dictionary. Matching is case-insensitive, and the returned word must be uppercase.

Input format
- Line 1: Source string.
- Line 2: Colon-separated dictionary words.

Output format
The longest matching word in uppercase, or NO-MATCH.

Constraints
- Source length: 1–100,000.
- Dictionary size: 1–10,000.
- Word length: 1–1000.
- Words contain English letters only.

Example

Input
abppplee
able:apple:ale:appeal:plea

Output
APPLE

Explanation: apple can be formed by selecting characters from the source in order. able and ale are also subsequences but shorter. appeal cannot be formed in the required order.

Problem 14. Find the Longest Substring Without Repeating Characters

A security system examines a sequence of characters from an access token. It needs to find the length of the longest contiguous substring in which no character appears more than once. Character comparison is case-sensitive.

Input format
Line 1: An alphanumeric string.

Output format
Print the maximum length of a substring containing no repeated characters.

Constraints

- String length: 1–200,000.
- Characters are English letters and digits.
- Case-sensitive comparison.

Example

Input
abcabcbb

Output
3

Explanation: abc, bca, and cab are examples of valid substrings of length three. No longer substring can avoid repeating a character.

Problem 15. Group Anagrams and Preserve Group Order

A dictionary-management system receives a list of words. Words that are anagrams of one another must be grouped together. Ignore letter case. Groups must be ordered according to the first appearance of any

member of the group, and words within each group must retain their original order. Output all words in uppercase.

Input format

- Line 1: Integer N.
- Line 2: N space-separated words.

Output format

Print each anagram group on a separate line, with words separated by spaces.

Constraints

- 1<=N <=20,000
- Word length: 1–100.
- Words contain English letters only.

Example

Input

6
eat tea tan ate nat bat

Output

EAT TEA ATE
TAN NAT
BAT

Explanation: eat, tea, and ate contain the same characters. The group appears first because eat occurs first in the input. The same principle applies to the other groups.

Problem 16. Find the Longest Repeated Substring

A log-analysis tool searches a string for the longest substring that occurs at least twice. The occurrences must start at different positions. Return the longest repeated substring. If several repeated substrings have the same maximum length, return the one whose first occurrence starts earliest.

Input format

Line 1: A string containing English letters and digits.

Output format

Print the longest repeated substring, or NO-REPEAT if no character repeats.

Constraints

- String length: 1–5000.
- Characters are case-sensitive.

- The two occurrences may overlap.

Example
Input
banana

Output
ana

Explanation: ana occurs twice in banana, starting at positions 2 and 4 when positions are counted from 1. It is longer than any other repeated substring.

Problem 17. Find the Minimum Window Containing All Required Characters

A security monitoring application receives a long log string and a short pattern of required characters. It must identify the shortest contiguous substring of the log that contains every character in the pattern, including repeated characters with their required frequencies.
If multiple windows have the same minimum length, return the earliest one. If no valid window exists, return NO-WINDOW.

Input format

- Line 1: Source string.
- Line 2: Required-character string.

Output format
Print the shortest valid substring, or NO-WINDOW.

Constraints

- Source length: 1–100,000.
- Pattern length: 1–1000.
- Strings contain English letters and digits.
- Matching is case-sensitive.

Example

Input
ADOBECODEBANC
ABC

Output
BANC

Explanation: BANC is the shortest substring containing A, B, and C. Although ADOBEC also contains all three characters, it is longer.



Problem 18. Find the Longest Common Substring Between Two Documents

A document-comparison system receives two strings and must find their longest common contiguous substring. Characters must appear consecutively in both strings and in the same order.
If several common substrings have the same maximum length, return the one whose occurrence starts earliest in the first string. If no common character exists, return NO-COMMON-SUBSTRING.

Input format

- Line 1: First string.
- Line 2: Second string.

Output format

Print the longest common substring, or NO-COMMON-SUBSTRING.

Constraints

- Each string length: 1–5000.
- Strings contain English letters and digits.
- Matching is case-sensitive.

Example

Input
ABABC
BABCA

Output
BABC

Explanation: BABC appears contiguously in both strings and has length four. No longer common contiguous substring exists.

Problem 19. Search a Dictionary Using Multiple Wildcard Types

A file-search application supports two wildcard symbols in its search pattern:

- ? matches exactly one character.
- * matches zero or more characters.

All other characters must match literally. The search is case-insensitive. Return all matching dictionary words in uppercase and in their original order.

Input format

- Line 1: Search pattern containing letters, ?, and *.
- Line 2: Colon-separated dictionary words.

Output format
Matching words separated by colons, or NO-MATCH.

Constraints
- Dictionary size: 1–5000.
- Word length: 1–100.
- Pattern length: 1–100.
- Words contain English letters only.
- The pattern may contain multiple wildcard symbols.

Example

Input
a*e
apple:axe:angle:able:ant

Output
APPLE:AXE:ANGLE:ABLE

Explanation: The pattern requires words to begin with a and end with e, with any number of characters between them. ant does not end in e.

Problem 20. Find the Minimum Cost to Transform One Word into Another

A spell-correction engine needs to transform one word into another using three operations:
- Insertion: Add a character.
- Deletion: Remove a character.
- Substitution: Replace a character.

Each operation has a configurable cost. Given the source word, target word, and the costs of these operations, calculate the minimum total cost required to transform the source into the target.

Input format
- Line 1: Source word.
- Line 2: Target word.
- Line 3: Three integers representing insertion, deletion, and substitution costs, respectively.

Output format
Print the minimum transformation cost as an integer.

Constraints
- Source and target lengths: 1–1000.
- Operation costs: 1–100.

- Words contain English letters only.
- Matching is case-sensitive.

Example
Input
cat
cut
1 1 2

Output
2

Explanation: Replace a with u. The substitution costs 2, and no other operation is needed. The minimum transformation cost is therefore 2.